

Chapitre 6 : Les sessions, les cookies et la redirection en PHP

I. Les sessions

I.1. Introduction

Les sessions sont un moyen de conserver des variables sur toutes les pages de votre site. Jusqu'ici, on a passer des variables de page en page via la méthode GET (en modifiant l'url : page.php?variable=valeur) et via la méthode POST (un formulaire).

Mais si on veut transmettre une ou plusieurs variables sur TOUTES les pages de son site, c'est vraiment difficile avec GET et POST... D'où l'invention des **sessions**.

I.2. Définition:

Une session est un mécanisme technique permettant de sauvegarder **temporairement sur le serveur** des informations relatives à un internaute. A chaque ouverture de nouvelle session, l'internaute se voit attribué un identifiant de session. Cet identifiant peut-être transmis soit en GET (PHPSESSID ajouté à la fin de l'url), POST ou Cookie (cookie sur poste client) selon la configuration du serveur. Les informations seront quant à elles transférées de page en page par le serveur et non par le client. Ainsi, la sécurité et l'intégrité des données s'en voient améliorées ainsi que leur disponibilité tout au long de la session. Une session peut contenir tout type de données : nombre, chaîne de caractères et même un tableau.

Contrairement à une base de données ou un système de fichiers, la session conserve les informations pendant quelques minutes. Cette durée dépend de la configuration du serveur mais est généralement fixée à 24 minutes par défaut. Le serveur crée des fichiers stockés dans un répertoire particulier.

Les sessions sont particulièrement utilisées pour ce type d'applications :

- Les espaces membres et accès sécurisés avec authentification.
- Gestion d'un caddie sur un site de vente en ligne.
- Formulaire éclatés sur plusieurs pages.

- Stockage d'informations relatives à la navigation de l'utilisateur (thème préféré, langues...).

Comment ça marche ?

1. Un visiteur se connecte. On demande à créer une session pour lui : PHP génère un numéro. Pour cela, on utilise la fonction `session_start()`. Ce numéro est souvent très gros et écrit en hexadécimal. Par exemple : `a02bbffc6198e6e0cc2715047bc3766f`

Ce numéro sert d'identifiant et est appelé "ID de session" (ou PHPSESSID).

PHP transmet automatiquement cet ID de page en page en utilisant un cookie ou via l'url (ex : `mapage.php?PHPSESSID=a02bbffc6198e6e0cc2715047bc3766f`)

2. A partir de là, on peut créer une infinité de variables de session. Par exemple : `$_SESSION['login']` contient le login du visiteur, `$_SESSION['password']` contient le mot de passe etc...

L'avantage, c'est que **le serveur conserve ces variables même lorsque la page PHP a fini d'être générée**. Ce qui veut dire que, quelle que soit la page de votre site, vous pourrez récupérer par exemple le login et le mot de passe du visiteur !

3. Lorsque le visiteur se déconnecte (il a cliqué sur un bouton "Déconnecter" ou est resté inactif trop longtemps), alors la session est fermée avec `session_destroy()`

Dans tous les cas où la sécurité de vos données est primordiale, vous devrez utiliser les sessions qui vous permettront de faire circuler différentes variables (comme un mot de passe par exemple) à travers les pages de votre site, tout en étant assez confortables à l'emploi.

Pour utiliser les sessions, différentes fonctions PHP s'offrent à nous:

Fonction	Signification
Session_start ()	Démarre une session
Session_register()	Enregistre une variable de session
Session_unregister()	Efface une variable de session
Session_is_registered()	Vérifie si une variable est déclarée pour la session en cours
Session_id()	Retourne l'id de la session en cours
Session_name()	Retourne le nom de la session en cours
Session_unset()	Detruit toutes les variables de la session en cours
Session_destroy()	Detruit la session en cours

I.3. Initialisation (et restauration) d'une session

Il ne faut pas oublier d'appeler `session_start()` sur chacune de vos pages avant d'écrire le moindre code HTML .Si vous oubliez `session_start()`, vous ne pourrez pas accéder aux variables superglobales `$_SESSION`.

Faites très attention : appelez `session_start` tout au début de vos pages PHP. Ne mettez la balise `<html>` qu'après, sinon vous aurez des problèmes avec votre session.

Après, vous pouvez utiliser les variables `$_SESSION` comme des variables normales.

Exemple :

```
<?php
session_start(); // On démarre la session AVANT toute chose

// créer quelques variables de session :
$_SESSION['prenom'] = 'Alia';
$_SESSION['nom'] = 'MAALOUL';
$_SESSION['age'] = 38;
// Maintenant que session_start est fait, on peut taper du code HTML
?>

<head>
    <title>Titre de ma page</title>
```

```

</head>
<body>
<p>
    Salut <?php echo $_SESSION['prenom']; ?> !<br />
    Tu es à l'accueil de mon site (index.php). Tu veux aller sur
une autre page ?
</p>
<p>
    <a href="mapage.php">Lien vers mapage.php</a><br />
    <a href="monscript.php">Lien vers monscript.php</a><br />
    <a href="informations.php">Lien vers informations.php</a>
</p>
</body>
</html>

```

Notez bien :

- on peut créer des variables de session n'importe où dans le code. La seule chose qui importe, c'est que le `session_start()` soit fait au tout début de la page.
- Les liens sont tous simples. C'est-à-dire qu'il ne faut pas transmettre le nom, prénom, âge du visiteur, ni de transmettre l'ID de session.

Quand votre site sera un peu gros et qu'il y aura plein de liens partout, vous apprécierez de savoir que PHP s'occupe tout seul de transmettre les variables. En effet, sur chacune des pages "mapage.php", "monscript.php", "informations.php" (et n'importe quelle autre page de votre site), vous retrouverez les variables `$_SESSION['prenom']`, `$_SESSION['nom']` et `$_SESSION['age']` !

Voici par exemple le code source de la page *informations.php* :

```

<?php
session_start(); // On démarre la session AVANT toute chose
?>

<html>
<head>
    <title>Titre de ma page</title>
</head>
<body>

    <p>Re-bonjour !</p>

```

```

<p>
    Je me souviens de toi ! Tu t'appelles <?php echo
$_SESSION['prenom'] . ' ' . $_SESSION['nom']; ?> !<br />
    Et ton âge ... Tu as <?php echo $_SESSION['age']; ?> ans,
c'est ça ?
</p>
</body>
</html>

```

En résumé, vous créez des variables de session comme vous créeriez des variables normales, sauf que vous mettez le préfixe `$_SESSION` devant pour que PHP sache qu'il doit "retenir" ces variables sur toutes les pages

I.4.Destruction d'une session

Comme cela a été évoqué plus haut, le serveur détruit lui même la session au bout d'un certain temps si la session n'a pas été renouvelée. En revanche, il est possible de forcer sa destruction au moyen de la fonction [`session\_destroy\(\)`](#). Cela permet par exemple aux webmasters de proposer une page de déconnexion aux membres loggués à leur espace personnel. Cependant, l'utilisation de `session_destroy()` seule n'est pas très "propre". Le code suivant présente une manière plus correcte de mettre fin à une session.

```

//Destruction propre et complète d'une session
<?php
    // Démarrage ou restauration de la session
    session_start();
    // Réinitialisation du tableau de session
    // On le vide intégralement
    $_SESSION = array();
    // Destruction de la session
    session_destroy();
    // Destruction du tableau de session
    unset($_SESSION);
?>

```

Pour être convaincu de la destruction de la session, il suffit juste d'essayer d'afficher le contenu du tableau de session au moyen de la fonction [`print\_r\(\)`](#).

Enfin, si vous voulez détruire la session du visiteur, vous pouvez faire un lien "Déconnexion" qui amène vers une page qui fait appel à la fonction `session_destroy()`. Quoiqu'il en soit, la session sera automatiquement détruite au bout d'un certain temps d'inactivité.

II. Les cookies

II.1. Introduction

Les cookies sont des fichiers que le serveur demande au client d'écrire sur son disque dur et qui permettent à un script de sauvegarder des variables liées à l'utilisateur.

On peut par exemple vouloir savoir combien de fois une personne est venue sur votre site pour lui proposer à la dixième visite de s'inscrire à une mailing-list.

Mais attention l'utilisation des cookies est à prendre avec des pincettes ! Plusieurs risques sont liés à l'utilisation des cookies:

- Le client peut refuser les cookies (A chaque visite vous lui direz "Bonjour, c'est la première fois que vous visitez ce site"... pas top)
- Le client peut avoir effacé les cookies et alors d'un jour à l'autre vous avez perdu les informations
- Le client peut changer de navigateur ou d'ordinateur d'une visite à l'autre

Les cookies ne doivent donc pas être utilisés pour des informations importantes. Une utilisation qui peut être faite consiste à remplir automatiquement un champ login qui a déjà été rempli précédemment par l'utilisateur (quoique maintenant cela est intégré aux navigateurs). Si le cookie est absent, l'utilisateur entrera manuellement le login.

II.2. Les cookies (avec PHP)

Il est important de savoir que la gestion du cookie doit se faire avant d'écrire une donnée dans le document HTML, attention aux espaces malencontreusement écrits au début du fichier.

En PHP, nous pouvons écrire et lire dans un cookie très simplement. La demande de génération du cookie se fait par la commande `setCookie()`.

Tous les cookies ainsi définis sont systématiquement retournés au script PHP et accessibles dans le tableau `$_COOKIE[]`. Pour affecter plusieurs valeurs à un seul cookie, il suffit que le nom du cookie soit suivi de `[ ]`. Voici maintenant les paramètres de la fonction `setCookie()`:

int setCookie(string \$name, string \$value, int \$expire, string \$path, string \$domain, int \$secure);

Seul le paramètre name est obligatoire.

- \$name est le nom du cookie, c'est une chaîne de caractères privée de ":", ",", " et " ".
- \$value est la valeur du cookie
- \$expire est la date d'expiration du cookie (tel que peuvent le retourner les fonctions `time()` ou `mktime()`), une fois la date atteinte le cookie est supprimé ou invalidé, s'il n'est pas spécifié, le cookie expire à la fin de la session (quand le navigateur est fermé).
- \$path est le répertoire dans lequel le cookie est valide. Les sous-répertoires sont également acceptés.
- \$domain est le domaine pour lequel le cookie est valide (ex: `phpfacile.com`)
- \$secure indique que le cookie ne doit être transmis que par une connexion sécurisée

Exemple:

```
setCookie("login", "toto", time()+3600);
```

Cette fonction va créer un cookie accessible par `$_COOKIE["login"]`, celui-ci est valide pendant 3600 secondes.

II.3. Exemple de script PHP utilisant des cookies (écriture du cookie)

Ici nous allons faire un caddie utilisant des cookies. Attention, c'est pas une bonne idée pour un vrai caddie car quelqu'un qui refuserait les cookies ne pourrait pas utiliser ce script. Le premier fichier que l'on appellera `cookie_init.php` servira à mettre à 0 les cookies.

```
<?php
setCookie("panier[pomme]", 0);
```

```

setCookie("panier[poire]", 0);
setCookie("panier[peche]", 0);
setCookie("panier[banane]", 0);
header("Location: cookie_ajout.php");
?>

```

On met à 0 tous les éléments du panier. On fait ensuite une redirection vers [cookie_ajout.php](#). *setCookie()* et *header()* requièrent tous les 2 qu'il n'y ait pas de données émises avant dans le script. S'il y avait eu un espace (ou d'autres caractères) devant **<?php**, il y aurait eu une erreur

II.4. Exemple de script PHP utilisant des cookies (lecture et mise à jour du cookie)

Ce second script est l'interface du site, on met à jour le cookie juste avant d'afficher l'état du panier.

```

<?php
    // Lecture du cookie
    $panier = $_COOKIE["panier"];
switch ($_GET["ajout"])
{
case "pomme":
$panier["pomme"]++;
setCookie("panier[pomme]", $panier["pomme"]);
break;
case "poire":
$panier["poire"]++;
setCookie("panier[poire]", $panier["poire"]);
break;
case "peche":
$panier["peche"]++;
setCookie("panier[peche]", $panier["peche"]);
break;
case "banane":
$panier["banane"]++;
setCookie("panier[banane]", $panier["banane"]);
break;
}

```

```

?>
<html>
<head>
<title>Le marché</title>
</head>
<body bgcolor="#FFFFFF">
<table border="4" cellspacing="4" cellpadding="4" align="center">
  <tr align="center">
    <td>Ajouter</td>
    <td>Votre commande</td>
  </tr>
  <tr align="center">
    <td><a href="cookie_ajout.php?ajout=pomme">1Kg pommes</a> (1E)</td>
  </tr>
  <tr align="center">
    <td><?echo $panier["pomme"]?>Kg pomme(s)</td>
  </tr>
  <tr align="center">
    <td><a href="cookie_ajout.php?ajout=poire">1Kg poire</a> (1,5E)</td>
  </tr>
  <tr align="center">
    <td><?echo $panier["poire"]?>Kg poire(s)</td>
  </tr>
  <tr align="center">
    <td><a href="cookie_ajout.php?ajout=peche">1Kg p&ecirc;che</a> (2E)
  </td>
    <td><?echo $panier["peche"]?>Kg p&ecirc;che(s)</td>
  </tr>
  <tr align="center">
    <td><a href="cookie_ajout.php?ajout=banane">1Kg banane</a> (1E)</td>
  </tr>
  <td><?echo $panier["banane"]?>Kg banane(s)</td>
  </tr>
</table>
<p align="center"><a href="cookie_init.php">vider mon panier</a></p>
<p align="center"><a href="cookie_calcul.php">calculer le total</a></p>
</body>
</html>

```

II.5. Exemple de script PHP utilisant des cookies (suite et fin)

Ce troisième script (*cookie_calcul.php*) calcule et affiche le prix total du panier.

```
<?php
    $panier = $_COOKIE["panier"];

$total = 0;
$total += $panier["pomme"]*1;
$total += $panier["poire"]*1.5;
$total += $panier["peche"]*2;
$total += $panier["banane"]*1;
?>

<html>
<head>
<title>Le total du panier</title>
</head>
<body bgcolor="#FFFFFF">
<p align="center">Le total de votre panier: <?echo $total?> E.</p>
<p align="center"><a href="cookie_ajout.php">Modifier mon panier</a></p>
<p align="center"><a href="cookie_init.php">Vider mon panier</a></p>
</body>
</html>
```

III. La redirection en PHP

Il existe des applications web pour lesquelles on souhaite rediriger le visiteur en fonction de paramètres. C'est le cas par exemple pour un script d'identification. Si l'internaute fournit les bons identifiants alors il est redirigé automatiquement vers son espace personnel, sinon il est renvoyé vers le formulaire d'authentification.

III.1. Présentation des redirections

Il y'a plusieurs manières de faire de la redirection HTTP par les pages Web : balises meta, script Javascript ou bien encore script PHP. Bien entendu, la dernière solution est de loin la meilleure et la plus propre car elle est entièrement gérée par le serveur, tandis que les deux autres sont réalisées du côté du client.

III.2. La fonction header()

Lorsque l'on souhaite créer une redirection avec PHP, on utilise une fonction permettant d'envoyer des entêtes de type *Location* (adresse). Pour cela, PHP dispose de la fonction header() qui se charge d'envoyer les entêtes passés en paramètre.

Règle importante : l'appel de cette fonction doit se faire avant tout envoi au navigateur (instruction echo, print, espace blanc, balise html...) sous peine de générer une erreur de type *Headers already sent by....* Cette erreur signifie que la page a déjà été envoyée au navigateur avant de vouloir envoyer des entêtes HTTP.

III.3. Script de redirection avec header()

La syntaxe de la redirection est simple. On passe en paramètre de la fonction header() une chaîne de caractères précisant le type d'entête (*Location*) et la page sur laquelle l'internaute doit être renvoyé.

Script de redirection

```
<?php
header('Location: http://www.votresite.com/pageprotegee.php');
exit();
?>
```

Note : l'instruction exit() qui succède la fonction header() permet de couper l'exécution du script car la redirection aura lieu immédiatement et le reste du code n'a pas d'intérêt à être interprété.

III.4. Exemple :

Soit un site qui possède une section membre où chaque membre devra se loguer avant de pouvoir y entrer. de plus, il s'agit toujours de ce même membre qui est connecté.

On aura alors une page contenant un formulaire permettant à notre visiteur de se connecter à une section membre (page index.htm) :

a. La page « index.htm »

```

<html>
<head>
<title>Formulaire d'identification</title>
</head>
  <body>
<form action="login.php" method="post">
Votre login : <input type="text" name="login">
<br />
Votre mot de passé : <input type="password" name="pwd"><br />
<input type="submit" value="Connexion">
</form>
  </body>
</html>

```

Observation:

D'après cette page, vous pouvez remarquer que lorsque le visiteur le remplira et qu'il cliquera sur le bouton de connexion, on se retrouvera au niveau de la page login.php avec une variable \$pseudo qui contiendra le login de notre visiteur ainsi qu'une variable \$pwd contenant son mot de passe ; variables qu'il faudra naturellement tester avant de démarrer notre session (car seuls les membres pourront accéder à notre espace membre, espace où l'on utilisera notre session).

b. La page « login.php »

```

<?php
// On définit un login et un mot de passe de base pour tester notre
exemple. Cependant, vous pouvez très bien interroger votre base de
données afin de savoir si le visiteur qui se connecte est bien membre
de votre site
  $login_valide = "moi";
  $pwd_valide = "lemien";
// on teste si nos variables sont définies
if (isset($_POST['login']) && isset($_POST['pwd'])) {

```

```

        // on vérifie les informations du formulaire, à savoir si le
pseudo saisi est bien un pseudo autorisé, de même pour le mot de
passe

        if ($login_valide == $_POST['login'] && $pwd_valide ==
$_POST['pwd']) {
            // dans ce cas, tout est ok, on peut démarrer notre session
                // on la démarre :)
                session\_start ();
// On enregistre les paramètres de notre visiteur comme variables
//de session ($login et $pwd) (notez bien que l'on utilise pas le $
//pour enregistrer ces variables)
                $_SESSION['login'] = $_POST['login'];
                $_SESSION['pwd'] = $_POST['pwd'];

// on redirige notre visiteur vers une page de notre section membre
                header ('location:page_membre.php');
            }
            else {
// Le visiteur n'a pas été reconnu comme étant membre de notre site.
// On utilise alors un petit javascript lui signalant ce fait
                echo '<body onLoad="alert(\'Membre non reconnu...\')">';
                // puis on le redirige vers la page d'accueil
                echo '<meta http-equiv="refresh"content="0;URL=index.htm">';
            }
        }
    else {
        echo 'Les variables du formulaire ne sont pas déclarées.';
    }
?>

```

Observation:

Remarquer que nous utilisons notre `session_start` avant tout code HTML.

c. La page « `page_membre.php` »

```

<?php
// On démarre la session (ceci est indispensable dans toutes les pages
// de notre section membre)

```

```

session\_start ();
// On récupère nos variables de session
if (isset($_SESSION['login']) && isset($_SESSION['pwd'])) {
    // On teste pour voir si nos variables ont bien été enregistrées
    echo '<html>';
    echo '<head>';
    echo '<title>Page de notre section membre</title>';
    echo '</head>';
    echo '<body>';
    echo 'Votre login est ' . $_SESSION['login'] . ' et votre mot de passe
est ' . $_SESSION['pwd'] . ' .';
    echo '<br />';
    // On affiche un lien pour fermer notre session
    echo '<a href="./logout.php">Déconnection</a>';
}
else {
    echo 'Les variables ne sont pas déclarées.';
}
?>

```

d. La page « logout.php »

```

<?php
// On démarre la session
session\_start ();
// On détruit les variables de notre session
session\_unset ();
// On détruit notre session
session\_destroy ();
// On redirige le visiteur vers la page d'accueil
header ('location: index.htm');
?>

```